It’s worth noting that if, instead of cataloguing all the coins, we would like to just count how many of them there are (without distinguishing their type) or computing the total value, we wouldn’t need reducers: each parallel process would just output its total, and then they would be added by a single central process.

The second catch is that there is no centralized entity that splits the work and distributes it evenly to the reducers, so one can get very busy while another stays without anything to do: going back to our story, for instance, while Daisy Duck, as we have seen, will have to worry about US currency, Gladstone Gander is assigned all rare coins from small Countries (lucky him - how could it be any different?) and thus the lists he gets are almost all empty, and he has to perform just a few additions.

13.2.3 There is More, Under the Hood

We have seen a few advantages of the MapReduce model, but there is also more to his success that can’t be seen at a high level. In chapter 7, talking about Caches and multi-threading, we have discussed locks and synchronization; every parallel computation with a shared state will need synchronization, being it to aggregate results, break down data and assign it to computing units (threads or machines) or just checking that the processing is complete.

The key advantage of MapReduce is that it intrinsically limits shared state to a minimum (by embracing functional programming concepts like immutability[7] and pure functions[8]), providing a programming paradigm, a way to